

Complexity Theory (2025-26)

Minor Solutions

Name:

Duration: 120 Minutes

Roll No:

Max. Marks: 36

Instructions:

1. There are 6 questions each of 6 marks, but they are not of equal difficulty level. So read all the question.
2. Make sure that the proofs/algorithms contain all the necessary details.
3. Be precise in your answers. Writing long gibberish will not get you any marks.
4. You are allowed to use any results we have seen in the class without proving them again.

1. Let M be a Turing machine and $\delta(q_{start}, \triangleright, \triangleright, \triangleright, \triangleright, \triangleright) = (q_{start}, \triangleright, \triangleright, \triangleright, \triangleright, S, S, S, S, S)$ is one of the entries of the transition function of M . Answer the following questions for M :

- a) Is there an input on which M halts? If yes, provide such an input. If not, justify your answer.
- b) How many tapes does M have? Justify your answer.

Solution: a) There is no input on which M halts as M stays on the same cells with the same content in the very first step due to the given entry of δ and hence enters into an infinite loop.

b) M has 5 tapes as the given entry of δ contains 5 symbols apart from the current state on LHS.

Marking Scheme: a) 2 for answering no input exists and 1 for reasoning.

b) 2 for answering 5 tapes and 1 for reasoning.

2. For each of the following statements, give two complexity classes that would become equal if the statements were proved true. Briefly justify your answers.

a) There exists a polynomial-time algorithm for $\overline{SAT}$.

b) There exists a logspace algorithm for $Path$.

Solution: a) One possible pair of two classes would be P and $coNP$. If a polynomial-time algorithm for $\overline{SAT}$ exists, then $coNP \subseteq P$ because $\overline{SAT}$ is a $coNP$ -complete language. Since we already know that $P \subseteq coNP$, we can conclude that $P = coNP$.

b) The two classes would be L and NL . If a logspace algorithm for $Path$ exists, then $NL \subseteq L$ because $Path$ is an NL -complete language. Since we already know that $L \subseteq NL$, we can say that $L = NL$.

Marking Scheme: a) 2 for giving two classes which will become equal and 1 for reasoning.

b) 2 for giving two classes which will become equal and 1 for reasoning.

3. Show that SAT has a logspace verifier (I asked you to think about it in the class, I hope you did).

That is, there exists a logspace TM M and a polynomial p such that

$$\phi \in SAT \iff \exists u \in \{0,1\}^{p(|x|)} \text{ such that } M(\phi, u) = 1.$$

Explain in detail the working of your logspace verifier.

Solution: Logspace verifier M on (ϕ, u) will check whether u is a satisfying assignment by checking whether all the clauses of ϕ are satisfied starting from the first clause. Additionally, to check that a

particular clause is satisfied it will note down all the literals of that clause one by one on the worktape and then check whether they are true as per u . More formally, suppose ϕ has n clauses and m variables. Then, logspace verifier M on (ϕ, u) will execute the following algorithm:

for $i = 1$ **to** n :

for every literal v of the i th clause:

if (v is getting value 1 according to u):

 move to the next clause

output 0 as i th clause is not getting satisfied

output 1 as all the clauses are getting satisfied

The above procedure takes only $O(\log n + \log m)$ space which is also logspace in the size of ϕ . The proof of correctness of the procedure is trivial and hence skipped.

Marking Scheme: Step marks will be given based on how far your answer is from a correct solution.

4. Prove that if $L = P$, then $PSPACE = EXP$. Write all the details in the proof.

(Hint: Padding argument.) (L is the logspace class.)

Solution: Since $PSPACE \subseteq EXP$ is already known, we will prove $EXP \subseteq PSPACE$ under the assumption that $L = P$. Let L be a language in EXP and M be its decider which runs in $O(2^{n^c})$ time. We will show that L is also in $PSPACE$. We first define L_{pad} below:

$$L_{pad} = \{x01^{2^{|x|^c}} \mid x \in L\}$$

We now describe the working of a polytime TM M' for L_{pad} on input y . M' will first check whether y is of the form $z01^{2^{|z|^c}}$ for some string z . And then simulates M on z and outputs $M(z)$. While simulation of M on z runs in time polynomial in the length of y , it's is non-trivial to show that checking y is in the form of $z01^{2^{|z|^c}}$ can also be done polynomial time (Hence, we will just assume it. If you are interested in knowing, you can meet me in in-person.) Now, since $L = P$, L_{pad} will also be in L . Suppose M'' is a logspace machine for L_{pad} .

We now define a $PSPACE$ machine N for L using M'' . N on input x :

1. First creates $y = x01^{2^{|x|^c}}$.
2. Simulates M'' on y and outputs $M''(y)$.

The only issue with N is that it cannot pad exponentially many 1s at the end of x as it is supposed to run in polynomial space. Hence, instead of actually creating a padded y , N will start simulation of M'' on y by giving bits of y only when they are required during the simulation. For that, N will maintain the tape head positions of the input tape of M'' which is manageable in polynomial space. Now, N 's space usage is space used by M'' , which is $O(\log(|y|)) = poly(|x|^c)$.

Marking Scheme: Step marking will be done. 1.5 marks will be deducted if you have simply padded exponentially many 1s at the end of x in the working of N .

5. In class, we saw a polynomial-time reduction from *Hampath* to *Hamcycle*. In this problem, you are required to give a polynomial-time reduction in the opposite direction, from *Hamcycle* to *Hampath*, and prove its correctness. Read the below definitions carefully before proceeding with the reduction.

- $\text{Hamcycle} = \{\langle G \rangle \mid G \text{ is an undirected graph with a hamiltonian cycle}\}$
- $\text{Hampath} = \{\langle G', s, t \rangle \mid G' \text{ is an undirected graph with a hamiltonian path from } s \text{ to } t\}$

Note: $\text{Hamcycle} \leq_p \text{Hampath}$ is already known as *Hampath* is **NP-complete** and *Hamcycle* is in **NP**.

In this problem, however, you need to give an explicit polynomial-time reduction from *Hamcycle* to *Hampath*. To avoid corner cases, you can assume that the input graph G contains at least three vertices.

Solution: You can read one possible reduction here: <https://math.stackexchange.com/a/1290804>. Although, it contains some unnecessary construction, the reduction is correct.

Marking Scheme: 3 for the reduction, 3 for the proof.

6. A language is called *unary* if every string in it is of the form 1^i (the string of i ones) for $i > 0$. Show that if there exists an **NP-complete** unary language then $\mathbf{P} = \mathbf{NP}$. (*Hint:* Try to design a polynomial-time algorithm for **SAT** using the fact that there is an **NP-complete** unary language.)

Solution: You can read a detailed outline here: <https://math.stackexchange.com/a/1416150>

Marking Scheme: Step marks will be given based on how far your answer is from a correct solution.